# ragent Research Manual

This guide explains how to use ragent's built-in research system to perform extensive, structured information gathering and report synthesis. The research system decomposes a topic into sub-queries, searches the web and your local project in parallel, gathers and vaults sources, then runs an adversarial quality pipeline before writing a structured `RESEARCH.md` report.

The same commands are available in three surfaces:

- **TUI** — `/research` slash commands in the interactive terminal interface
- **CLI** — `ragent research <subcommand>` from the shell
- **HTTP API** — `GET/POST/PUT/DELETE /research` endpoints on the ragent server

This document is written for the terminal interface but covers all three.

> **Scope:** `/research` slash commands, the Hyperresearch adversarial pipeline, source vaulting, local/spec cross-referencing, open-access recovery, templates, and the built-in markdown viewer. For web-search engine configuration and diagnostics, see `/websearch` in the TUI (`/websearch help`).

---

## 1. Purpose and capabilities

The research system is designed for situations where you need more than a single web search — when you need a **structured, cited, traceable report** that synthesises multiple sources, flags contradictions, and stands up to adversarial quality review.

### What it does

- **Topic decomposition** — breaks a research topic into focused sub-queries using either a deterministic heuristic planner or an LLM-backed planner.
- **Multi-engine web search** — queries DuckDuckGo, Brave, OpenAlex (scholarly works), and Wikipedia in parallel, plus optional API-backed engines (LangSearch, Tavily, Perplexity, Exa) when configured.
- **Local project cross-referencing** — scans your codebase and project files for relevant content, scoring matches by keyword overlap with the topic.
- **Prior-spec cross-referencing** — reads existing specifications under `specs/<id>/` and weaves them into the research corpus.
- **Source vaulting** — persists every captured source (body text + metadata) to a SQLite-indexed vault so future runs can reuse sources without re-fetching.
- **Adversarial quality pipeline** — the `full` and `dissertation` tiers run a 16-step deterministic pipeline including contradiction detection, loci analysis, corpus critique, gap-fill fetching, triple-draft synthesis, and citation verification.
- **Multiple output formats** — report (default), executive summary, comparison table, source bibliography, and IMRaD scientific format.
- **Open-access recovery** — for short scholarly sources, queries Unpaywall and Europe PMC for legal full-text copies.
- **Seed sources** — start research from a URL, a local file (PDF, DOCX, ODT, markdown, etc.), or an explicit topic string.
- **Resumability** — the source vault and run manifest allow subsequent runs with the same name to skip already-completed gathering.

**What it does not do**

- It does not replace interactive coding tasks — it produces research documents, not code changes.
- It does not crawl an entire website (use `mf_crawl` for that); it fetches individual pages returned by search engines.
- It does not perform real-time monitoring; each `/research create` is a one-shot gathering session.

---

## 2. Quick start

Open ragent and run:

```
/research create rust-async "Rust async/await patterns"
```

The TUI will:

1. Create a new research item under `research/rust-async/`.
2. Decompose the topic into sub-queries, search the web (DuckDuckGo, Brave, OpenAlex, Wikipedia, plus any configured API engines), optionally scan local project files and specs, and synthesize a structured report.
3. Stream progress into the message window and status bar.
4. Write the final document to `research/rust-async/RESEARCH.md`.

When it finishes, open the result with:

```
/research open rust-async
```

A typical `RESEARCH.md` contains:

- YAML frontmatter (title, topic, status, timestamps, source counts, tier, format, open-access disclosure)
- Executive summary
- Top implications
- Open questions
- Findings (with inline `[#N]` citations)
- Findings relationship diagram
- Cross-references to local files and specs
- Contradiction graph (full/dissertation tier)
- Source tensions (full/dissertation tier)
- Numbered references index

---

## 3. What research items are

A research item is a self-contained folder under `research/<name>/`:

```
research/
   INDEX.md
   rust-async/
       RESEARCH.md          # final report (frontmatter + markdown body)
       sources/
           web-01.md
```

```
web-02.md
web-03.md
local-01.md
spec-01.md
```

- `RESEARCH.md` is the deliverable. It contains an executive summary, findings, implications, diagrams, cross-references, open questions, and a numbered references index.
- `sources/` holds the captured body text of every web, local, and spec source so each citation can be traced back to the original text.
- `research/INDEX.md` is a derived table of all items, regenerated automatically on every create/delete/archive.
- `.ragent/research_vault/<run_tag>/` holds the SQLite-indexed source vault with raw content files for resumability.

**Naming rules**

Research names must be 3–64 characters, start with a lowercase ASCII letter, and contain only lowercase letters, digits, and hyphens. This rule prevents path traversal and keeps directory names URL-safe.

Valid names: `rust-async`, `openai-reasoning`, `q3-review`, `vector-db-2024`

Invalid names: `Rust-Async` (uppercase), `rust_async` (underscore), `1topic` (starts with digit), `a` (too short), `a..b` (dots not allowed)

---

## 4. The `/research` slash command family

All commands are entered in the TUI input box. The CLI equivalents use `ragent research <subcommand>`.

### 4.1 `/research help`

Show the command syntax, flags, and formats. Use this whenever you forget a flag.

```
/research help
```

### 4.2 `/research create`

Run a research session and write `RESEARCH.md`. This is the primary command.

```
/research create <name> [topic]
  [--from-url <URL>] [--from-file <PATH>]
  [--iterations N] [--depth shallow|standard|deep]
  [--tier light|full|dissertation]
  [--mode normal|supervisor|competitive]
  [--format report|executive-summary|comparison-table|source-bibliography|imrad]
  [--sources-dir <path>] [--template <name>]
  [--fetch-concurrently N] [--local-concurrently N]
  [--fetch-timeout-secs N]
  [--web-time N] [--web-phase-timeout-secs N] [--local-phase-timeout-secs N]
  [--search-max-retries N] [--search-retry-base-delay-ms N]
  [--search-circuit-breaker-threshold N]
```

```
[--max-web-results N] [--max-search-calls N]
[--use-local] [--use-specs] [--use-low-relevance] [--no-papers] [--use-pdf]
```

`--mode competitive` runs the multi-researcher competitive pipeline and implicitly defaults `--format` to `comparison-table`, so it need not be passed explicitly.

If the first argument after `/research create` is not a recognised subcommand, the parser treats the whole line as `<name> <topic>`; this lets you type quickly when the name is valid.

**Re-running a research item:** `/research update <name>` replays the item's recorded invocation line and re-runs the full pipeline, overwriting `RESEARCH.md` and its associated files (`CORPA.md`, `sources/`, index) with freshly gathered results. The original invocation is preserved on the replayed run. The same replay is available from the CLI (`ragent research update <name>`) and over HTTP (`PUT /research/{name}`).

**Quick syntax note:** The `--from-url` flag can be repeated to seed multiple pages. The `--from-file` flag accepts PDF, DOCX, XLSX, PPTX, ODT, ODS, ODP, TXT, and MD files. When neither a topic nor a seed source is supplied, the command errors.

**Examples** Basic research with default settings (full tier, standard depth, report format):

```
/research create rust-async "Rust async/await patterns"
```

Deep research with full adversarial pipeline:

```
/research create openai-reasoning "OpenAI o1-style reasoning" --tier full --depth deep
```

Quick light-tier lookup for a fast answer:

```
/research create quick-check "Is Rust 1.85 stable" --tier light
```

Local-only research (no web search, scan project files only):

```
/research create local-only "Project error handling" --use-local --no-papers
```

Seed from a web page (topic derived from page content):

```
/research create from-page --from-url https://example.com/article
```

Seed from multiple web pages:

```
/research create multi-source --from-url https://example.com/a --from-url https://example.com/b
```

Seed from a local document:

```
/research create from-doc --from-file docs/design.md --use-local
```

Comparison table format for evaluating options:

```
/research create benchmark "Comparison of vector databases" --format comparison-table
```

IMRaD scientific report format:

```
/research create paper-review "Attention Is All You Need" --format imrad --tier full
```

Dissertation tier for long-form deliverable with chapter partitioning:

```
/research create thesis "Survey of multi-agent orchestration frameworks" --tier dissertation --de
```

Include project files and specs in the research:

```
/research create rust-async "Rust async patterns" --use-local --use-specs
```

Allow PDF web sources (skipped by default):

```
/research create pdf-research "Machine learning benchmarks" --use-pdf
```

Keep low-relevance sources that would normally be filtered:

```
/research create broad-scan "Emerging programming languages 2024" --use-low-relevance
```

Customise fetch concurrency and timeouts for slow networks:

```
/research create slow-net "Distributed consensus algorithms" --fetch-concurrently 4 --fetch-timec
```

Use a custom template:

```
/research create quarterly "Q3 performance review" --template quarterly
```

Override iterations explicitly (activates iterative planner/critic loop):

```
/research create deep-iter "Rust memory safety" --iterations 7 --depth deep
```

### 4.3 `/research list`

List research items, newest first.

```
/research list
/research list --all   # include archived items
```

The default output is a fixed-width `NAME/TITLE/STATUS/CREATED/MODIFIED` table; JSON is available behind a `--json` flag (CLI and HTTP).

### 4.4 `/research show <name>`

Print metadata for one item: title, status, timestamps, topic, and all captured sources with their provenance (type, URL/path, title, capture time, and open-access recovery note if applicable).

```
/research show rust-async
```

### 4.5 `/research open <name>`

Open the rendered `RESEARCH.md` in the TUI's markdown overlay. See section 11 for viewer controls.

```
/research open rust-async
```

### 4.6 Concepts section in `/research create`

When `/research create` runs with an LLM configured, a concept-extraction step runs automatically after synthesis. The gathered corpus is sent to the model as a context-bounded payload (each source block headed `--- [#N] Title ---`, where `N` is the source's position in the References Index), and the numbered concept list it returns is embedded in `RESEARCH.md` as a `## Concepts` section directly above `## Findings`:

```
## Concepts

### 1. Semantic Scholarly Search
```

```
**Definition:** Retrieval that understands research questions rather than
exact keywords.

**Key Evidence:**
- hybrid retrieval across 200M+ papers ([#6])
```

`[#N]` citations resolve against the References Index at the bottom of `RESEARCH.md`, exactly like the Findings citations. The section is omitted when no LLM is configured, the extraction fails, or the model returns no concept sections; the step is visible in the progress output as a `concepts` step (started / completed / skipped / failed).

### 4.7 `/research cluster <name>`

Extract up to 20 core concepts from the captured sources of an existing research item and write them to `research/<name>/CONCEPTS.md`. The command reads every `web-*.md` and `local-*.md` source in the item's `sources/` directory, builds a context-window-aware payload, and asks the active model for a structured concept list. Sources are truncated individually if the combined payload exceeds the model's context budget.

```
/research cluster rust-async
```

The output is a markdown file with a `# Concepts` heading, followed by one numbered section per concept (`## 1. Concept Name`, `## 2. Next Concept`, …) containing a short definition and key evidence bullets. Inline citations use the same `[#N]` style as `RESEARCH.md`; each concept that cites captured sources gains a `**WebSources:**` block listing those sources in the same format as the `**Sources:**` lists under `## Findings` in `RESEARCH.md`:

```
## 3. Semantic Scholarly Search

**Definition:** Retrieval that understands research questions rather than
exact keywords.

**Key Evidence:**
- hybrid retrieval across 200M+ papers ([#6])

**WebSources:**
- [6] Paperguide: AI Search - [https://example.com/...](https://example.com/...) (published 2025
```

The companion is rewritten on every run and uses the same source indices as `RESEARCH.md`. Concept headings are renumbered sequentially on write, even if the model emits them out of order or unnumbered.

### 4.8 `/research search <query>`

Full-text search across all `RESEARCH.md` files. Returns matching item names, titles, and snippets. The search returns up to 25 results.

```
/research search "vector database"
/research search async runtime tokio
```

### 4.9 `/research delete <name> --yes`

Permanently delete a research item and its `sources/` directory. The `--yes` flag is required; without it the command prompts and refuses.

```
/research delete rust-async --yes
```

#### 4.10 `/research archive <name>`

Mark an item as `archived`. Archived items are hidden from default `list` output but kept on disk. Use `/research list --all` to see archived items.

```
/research archive rust-async
```

#### 4.11 `/research continue <name> [message]`

Resume an in-progress item. Loads state and appends a follow-up sub-question to the plan. The optional message is added to the plan as a new sub-question. The saved state file is written back unchanged when there is no follow-up — an item's `InProgress`/`Complete` status lives in the `RESEARCH.md` frontmatter, not the state file.

```
/research continue rust-async "Also cover tokio vs async-std comparison"
```

---

## 4A. Web-phase deadline

As of v1.0.76, `/research create` caps the web-gathering phase at 60 seconds by default. Use `--web-time N` (or `--web-phase-timeout-secs N`) to change the deadline; set `--web-time 0` to disable it.

When the deadline elapses, the run ingests everything gathered so far and continues to analysis/synthesis with the partial source set. No new search or fetch is started after the deadline, so the phase cannot run unbounded — the worst-case overshoot is the completion of fetches already in flight, each capped by `--fetch-timeout-secs` (default 30 s).

The TUI status bar shows a live `web:M:SS` countdown while the web phase is active, and a single quantified notice is added to the research progress message when the deadline is reached, for example:

```
**Web phase deadline reached** - 7 source(s) captured before timeout; proceeding to synthesis
```

---

## 5. Tiers: light, full, dissertation

The `--tier` flag selects how much adversarial quality assurance runs after gathering. The default is `full`.

| Tier | Pipeline steps | Best for | Min sources (vault skip) |
|---|---|---|---|
| `light` | decompose, width sweep, evidence digest, triple draft, synthesize, cite check, polish | Quick orientation, fast answers | 3 |
| `full` | All 16 Hyperresearch steps (see section 12) | Thorough reports, decision documents | 8 |

| Tier | Pipeline steps | Best for | Min sources (vault skip) |
|---|---|---|---|
| `dissertation` | `full` plus chapter partitioning into up to 12 chapters | Long-form deliverables | 15 |

**When to use each tier**

- `light` — when you need a quick answer and do not need contradiction analysis or deep evidence auditing. Runs in seconds to a minute.
- `full` (default) — when you need a thorough, decision-quality report with contradiction detection, evidence ranking, and citation verification. Runs in minutes.
- `dissertation` — when you need a long-form, chaptered deliverable suitable for extended writing. The pipeline partitions the topic into up to 12 chapters and runs the full adversarial pipeline per chapter.

```
/research create quick "What is WebAssembly" --tier light
/research create decision "Rust vs Go for microservices" --tier full
/research create survey "History of functional programming" --tier dissertation
```

---

## 6. Depth and iterations

`--depth` selects source and iteration budgets. The default is `standard`.

| Depth | Iterations | Sources per sub-question | Concurrency |
|---|---|---|---|
| `shallow` | 1 | 2 | 2 |
| `standard` (default) | 3 | 3 | 4 |
| `deep` | 5 | 5 | 6 |

**Web volume is bounded by depth by default.** Unless `--max-web-results` is passed explicitly, the effective web-source budget for gather passes is derived from the selected depth (shallow 6 / standard 9 / deep 15 sources), so `--depth shallow` actually limits search/fetch volume instead of collapsing into the 500-source ceiling. `--max-search-calls N` additionally places a hard, run-scoped cap on the total number of web-search calls a run may issue, shared across every supervisor/competitive researcher and gather pass; when the cap is reached, remaining sub-queries are skipped and the run proceeds with the sources gathered so far.

**Iteration override**

`--iterations N` overrides the default iteration count. When set (or when `--depth deep` is used), the iterative research engine activates and runs a planner/critic loop:

1. The **planner** decomposes the topic into sub-questions (using the heuristic planner by default, or the LLM planner when available).
2. Each iteration gathers sources for pending sub-questions concurrently (up to `max_concurrency`).
3. The **critic** evaluates the gathered evidence, scores quality, and identifies evidence gaps.
4. Gaps generate follow-up queries for the next iteration.

5. An **adaptive stopper** decides whether to continue: it stops when the critic score is high enough, all sub-questions are answered, or the maximum iteration count is reached.
6. If no progress is made (no new sources, no answered questions) and `force_deeper` is not set, the loop stops early.

#### Local source budgets

The depth also controls how many local files are captured:

| Depth | Max local sources |
|---|---|
| `shallow` | 5 |
| `standard` | 10 |
| `deep` | 20 |

```
/research create fast "Quick overview of REST APIs" --depth shallow
/research create balanced "REST vs GraphQL vs gRPC" --depth standard
/research create thorough "Comprehensive guide to API design" --depth deep --iterations 8
```

---

## 7. Output formats

`--format` changes the skeleton and final layout of `RESEARCH.md`.

| Format | Layout |
|---|---|
| `report` (default) | Topic, Search Queries, Executive Summary, Top 10 Implications, Open Questions, Findings, Findings Relationship Diagram, Cross-References, References Index |
| `executive-summary` | One-page summary of the same content |
| `comparison-table` | Topic, Research Brief, Executive Summary, Comparison Criteria, Comparison Table, Entity Profiles, Findings, References Index |
| `source-bibliography` | Bibliography of all captured sources |
| `imrad` | Scientific/technical report: Abstract, Introduction, Methods, Results, Discussion, References Index |

The format is recorded in the item frontmatter and can be reopened later with `/research open`.

#### CORPA.md companion

Quality-assurance renders (Contradiction Graph, Loci Analysis, Depth Investigation, Cross-Locus Reconcile, Source Tensions, Synthesis Audit, Corpus Critic) are written to `research/<name>/CORPA.md` rather than inline in `RESEARCH.md`. The companion file is created with the skeleton and rewritten on every full write; its `## Sources Reference` section repeats the References Index table so `[#N]` indices resolve in either document.

**Format aliases**

The parser accepts several aliases for each format:

| Format | Accepted values |
|---|---|
| `report` | `report`, `default` |
| `executive-summary` | `executive-summary`, `executive_summary`, `summary` |
| `comparison-table` | `comparison-table`, `comparison_table`, `comparison` |
| `source-bibliography` | `source-bibliography`, `source_bibliography`, `bibliography` |
| `imrad` | `imrad`, `im-rad`, `scientific` |

**When to use each format**

- **`report`** — general-purpose research report with findings and implications. The default for most use cases.
- **`executive-summary`** — when you need a one-page brief for decision-makers.
- **`comparison-table`** — when evaluating options (tools, frameworks, approaches) side by side.
- **`source-bibliography`** — when you need a clean bibliography of all sources without narrative findings.
- **`imrad`** — for scientific or technical reports following the Introduction/Methods/Results/Discussion structure.

```
/research create exec-brief "Q4 security posture" --format executive-summary --tier light
/research create tool-comparison "PostgreSQL vs MySQL vs SQLite" --format comparison-table
/research create bib "Sources on transformer architectures" --format source-bibliography
/research create sci-report "Reproducing GPT-3 results" --format imrad --tier full
```

---

## 8. Seed sources and local context

### 8.1 Seed a research from a URL

```
/research create from-url --from-url https://example.com/article
```

The page is fetched first; its extracted body is used to derive the topic, then normal web search runs on that topic. The fetched page is also captured as the first primary web source. `--from-url` can be repeated for multiple seed pages:

```
/research create multi-seed --from-url https://blog.example.com/post1 --from-url https://blog.exa
```

When a topic is also supplied, both the topic and the fetched page content are used. When no topic is supplied, the topic is derived from the cleaned page body (via readability extraction), falling back to the page title, then the URL.

### 8.2 Seed from a local file

```
/research create from-doc --from-file docs/design.md --use-local
```

The file body is extracted as the primary `Other` source and used to derive the topic. Supported file formats:

- PDF (`.pdf`)

- Microsoft Office (`.docx`, `.xlsx`, `.pptx`)
- LibreOffice/ODF (`.odt`, `.ods`, `.odp`)
- Plain text (`.txt`)
- Markdown (`.md`)

When no topic is supplied, a concise topic is derived from the extracted text. The normal web-search phase still runs using the derived topic.

```
/research create from-pdf --from-file papers/attention-is-all-you-need.pdf --format imrad --tier
/research create from-docx --from-file specs/design.docx --use-specs
```

### 8.3 Include project files and specs

```
/research create rust-async "Rust async patterns" --use-local --use-specs
```

- `--use-local` enables scanning the project root and any `--sources-dir` for files matching the topic keywords. Files are scored by keyword overlap and the top results are captured with excerpts.
- `--use-specs` cross-references prior specifications under `specs/<id>/`. Each spec's SPEC.md is scanned for relevance and captured as a spec source.

### 8.4 Extra sources directory

```
/research create with-extras "Project architecture" --use-local --sources-dir docs/architecture
```

`--sources-dir <path>` adds an additional directory to the local scan. Files in this directory are scored alongside project-root files.

---

## 9. Tuning gathering behavior

These flags control the performance and resilience of the gathering phases.

| Flag | Default | What it does |
|---|---|---|
| `--fetch-concurrency N` | 10 | Number of candidate pages fetched in parallel during web gathering. `0` is clamped to `1`. |
| `--local-concurrency N` | 8 | Parallel local-file scoring tasks. `0` is clamped to `1`. |
| `--fetch-timeout-secs N` | 30 | Per-page fetch timeout. Pages exceeding this are treated as fetch failures. |
| `--web-time N` | 60 | Wall-clock timeout for the entire web phase (alias of `--web-phase-timeout-secs`). When the deadline passes, everything gathered so far is ingested and the run continues to analysis/synthesis with the partial source set. `0` disables the deadline. No new search or fetch is started after the deadline; the worst-case overshoot is bounded by the in-flight fetches' own `--fetch-timeout-secs`. |
| `--local-phase-timeout-secs N` | none | Wall-clock timeout for the entire local phase. Aborts if exceeded. |

| Flag | Default | What it does |
|---|---|---|
| `--search-max-retries` N | 2 | Retries per failed sub-query. `0` disables retries. |
| `--search-retry-base-delay-ms` N | 200 | First retry delay in ms, doubled each retry (200, 400, 800…). |
| `--search-circuit-breaker-threshold` N | 3 | Consecutive search failures before circuit breaker opens. `0` disables. |
| `--max-search-calls` N | derived from `--depth` | Hard, run-scoped cap on total web-search calls, shared via `Arc` across every supervisor/competitive researcher and gather pass. A run-scoped query cache memoises identical sub-queries so parallel researchers reuse cached hits instead of re-issuing paid calls. |
| `--max-web-results` N | derived from `--depth` (shallow 6 / standard 9 / deep 15) | Cap on the web-source budget. |
| `--use-low-relevance` | off | Keep sources that would normally be filtered out as low-relevance. |
| `--no-papers` | off | Disable scholarly backends (OpenAlex) so only general web results are captured. |
| `--use-pdf` | off | Allow PDF documents from web search or `--from-url` to be captured as sources. |

GitHub `blob/` file-view URLs (`github.com/owner/repo/blob/...`) are rewritten to `raw.githubusercontent.com` before fetching and non-HTML content bypasses the readability gate, so GitHub file URLs no longer fail gathering with a readability error.

**Performance tuning examples**

For a slow network, reduce concurrency and increase timeouts (including the 60-second web phase deadline):

```
/research create slow-net "Distributed systems" --fetch-concurrently 4 --fetch-timeout-secs 60 --
```

For a fast network with many expected results, increase concurrency:

```
/research create fast-net "Rust ecosystem 2024" --fetch-concurrently 20
```

To prevent a slow local filesystem scan from blocking, set a local phase timeout:

```
/research create big-project "Codebase architecture" --use-local --local-phase-timeout-secs 120 -
```

To make search more resilient against transient failures:

```
/research create resilient "Edge computing platforms" --search-max-retries 5 --search-retry-base-
```

To disable the circuit breaker entirely (keep searching regardless of failures):

```
/research create no-breaker "Niche topic with sparse results" --search-circuit-breaker-threshold
```

## 10. The Hyperresearch pipeline in `full` tier

The `full` (and `dissertation`) tiers run an adversarial, deterministic quality pipeline after gathering. The pipeline has 16 steps, each of which produces observable progress events:

1. **Decompose** — break the topic into sub-queries using the planner.
2. **Width sweep** — parallel web search across all sub-queries.
3. **Contradiction graph** — detect opposing source claims on the same dimension and rank them by strength. The graph uses polarity dimensions (e.g. positive vs negative sentiment) to identify pairs of sources that make contradictory claims. Each contradiction is scored: base 30 + 20 per overlapping dimension, capped at 100.
4. **Loci analysis** — find recurring research dimensions (loci) across the corpus. Each locus represents a key claim or dimension that multiple sources address.
5. **Depth investigation** — classify each locus as `surface`, `moderate`, or `deep` based on how many sources address it.
6. **Cross-locus reconcile** — surface dimensions that share common sources, identifying where evidence overlaps and where it diverges.
7. **Source tensions** — list contradictions, shallow evidence, and isolated sources that lack corroboration.
8. **Corpus critic** — audit evidence quality and coverage. Identifies sub-questions with no sources, weak evidence, and coverage gaps.
9. **Evidence digest** — rank claims from strongest to weakest support, marking contested ones. Strong claims have multiple independent sources; weak claims rely on a single source or face contradictions.
10. **Triple draft** — produce three deterministic candidate summaries:
    - **Consensus** — the majority view across sources.
    - **Skeptical** — a cautious view that flags contested claims.
    - **Gap-aware** — highlights what the evidence does not cover.
11. **Synthesize** — combine the triple drafts into a final narrative draft, weighted by evidence strength and critic scores.
12. **Critics** — four internal critic subagents score the draft on:
    - **Coverage** — are all sub-questions addressed?
    - **Logic** — do findings follow from evidence?
    - **Evidence** — are claims properly supported?
    - **Readability** — is the draft clear and well-structured?
13. **Gap-fill fetch** — issue targeted follow-up queries for evidence gaps identified by the corpus critic. New sources are fetched and added to the corpus.
14. **Patcher** — apply deterministic surgical revisions to the draft based on audit and critic output. Fixes citation mismatches, clarifies contested claims, and strengthens weak evidence.
15. **Cite check** — verify that every `[#N]` citation marker in the draft points to a gathered source. A failure blocks report shipment and emits a `CITATION_VERIFICATION_FAILED` diagnostic.
16. **Polish** — clean whitespace and control characters, finalise formatting, and score the final draft.

The `dissertation` tier adds a **ChapterPartition** step before all others, dividing the topic into up to 12 chapters. Each chapter then runs the full pipeline independently.

Each step appears in `RESEARCH.md` when it produced results. Steps that produce no output (e.g. no contradictions found) are omitted from the final report. The run manifest tracks every step's lifecycle (pending, in-progress, completed, skipped, failed) and is persisted for resumability.

---

## 11. Reading results in the TUI

`/research open <name>` opens the markdown viewer overlay.

| Key / mouse | Action |
|---|---|
| `Up` / `Down` | Scroll one line |
| `PageUp` / `PageDown` | Scroll one page |
| `Ctrl+PageUp` / `Ctrl+PageDown` | Jump to start / end |
| `Mouse wheel` | Scroll inside the overlay when the cursor is over it |
| `Click outside` or `Esc` | Close the overlay |

The viewer renders headings, lists, code blocks, tables, and inline formatting. Tables (including the references index) are preserved as rows.

---

## 12. Following live progress

When a research session runs, the TUI shows progress in three places:

1. **Status bar** — a transient line such as `research: rust-async -- web -- running`. It updates per phase and becomes `research: rust-async complete -- 12 sources` when finished. While the web phase is active, the top-right status segment also shows a live `web:M:SS` countdown from the phase deadline (see below).
2. **Message window** — a pinned `Research Progress -- rust-async` log lists each phase (setup, web, local, specs, synthesize, assemble, finalize) and audit results. Captured web sources are aggregated into a per-engine summary table (counts by media type and per-language article counts) instead of one line per URL, keeping the window compact while sources stream in.
3. **Log panel** — every progress event is logged at `Info` level.

Research runs in the background; you can continue typing or start other work while it gathers.

#### Web-phase deadline countdown and timeout notice

When a `/research create` run enters its web-gathering phase, the TUI status bar shows a live countdown in the top-right wait segment, for example `web:0:45`. The countdown is computed from a stored wall-clock deadline at render time, so it stays accurate even if the event loop skips redraws or receives a burst of events (FR-010, FR-013). It updates at least once per second while the web phase is active and is removed as soon as the phase completes, times out, or fails (FR-011).

When the configured `--web-time` deadline elapses, the run continues with the sources already captured. A single, quantified notice is added to the research progress message, for example:

```
**Web phase deadline reached** - 7 source(s) captured before timeout; proceeding to synthesis
```

This notice is rendered at most once per run (FR-012). The underlying `PhaseTimedOut` diagnostic is also emitted exactly once per gather phase, carrying the effective deadline and the final captured count (FR-004).

Set `--web-time 0` to disable the deadline and allow the web phase to run to natural completion (FR-007).

#### Progress events

The session emits structured events that the TUI renders:

- **Phase** — phase transitions (setup, web, local, specs, synthesize, assemble, finalize).
- **QueriesDecomposed** — the sub-queries generated by the planner.
- **WebCaptured** — each web source captured (URL, title, search engine, body preview, language, media type, OA recovery info). The TUI folds these into the per-engine capture summary table; the log panel keeps a one-line-per-URL record.
- **FromUrlBodyPreview** — preview of a `--from-url` seed page body.
- **FromFileBodyPreview** — preview of a `--from-file` seed document body.
- **LocalCaptured** — each local file captured (path, relevance score).
- **SpecCaptured** — each spec captured (spec ID).
- **IterationCompleted** — iteration number and critic score.
- **CriticResult** — critic quality score and identified evidence gaps.
- **VerificationResult** — citation verification pass/fail and issues.
- **FollowUpQueries** — gap-driven follow-up queries for the next iteration.

---

## 13. Web search engines

By default, research uses `mf_search`, which queries DuckDuckGo, Brave, OpenAlex, and Wikipedia in parallel. Optional API-backed engines can be added in `ragent.json`:

```
{
    "langsearch_api_key": "...",
    "tavily_api_key": "...",
    "perplexity_api_key": "...",
    "exa_api_key": "...",
    "openalex_email": "you@example.com"
}
```

Use the TUI diagnostics to check availability:

```
/websearch show   # enabled / in-use / failed per engine
/websearch test   # live probe each configured engine
```

The `--no-papers` flag disables OpenAlex for runs where scholarly results are not wanted. This is useful when researching non-academic topics where scholarly papers would add noise.

#### Keyless backends

These backends require no API keys and run by default:

- **DuckDuckGo** — general web search
- **Brave** — general web search
- **OpenAlex** — scholarly works catalog (set `openalex_email` for the polite pool)
- **Wikipedia** — English Wikipedia encyclopedia summaries

#### API-backed backends

These require API keys configured in `ragent.json`:

- **LangSearch** — `langsearch_api_key`
- **Tavily** — `tavily_api_key`
- **Perplexity** — `perplexity_api_key`
- **Exa** — `exa_api_key`

---

## 14. Open-access recovery

For scholarly sources, ragent can attempt to recover a legal full-text copy via Unpaywall and Europe PMC when the fetched body is shorter than the configured minimum (default: 1000 characters).

Enable it in `ragent.json`:

```
{
    "research": {
        "open_access_recovery": true,
        "contact_email": "you@example.com",
        "oa_min_full_text_chars": 1000
    }
}
```

`contact_email` is required by Unpaywall's terms of service. When a source is recovered from an OA copy, the references index notes the source URL, OA source (Unpaywall or Europe PMC), version, and license.

The recovery process:

1. The web gatherer fetches a scholarly source page.
2. If the body is shorter than `oa_min_full_text_chars`, the gatherer queries Unpaywall for an OA copy using the source URL.
3. If Unpaywall returns a result, the OA copy is fetched.
4. If Unpaywall does not have a copy, Europe PMC is queried as a fallback.
5. The recovered full text replaces the short body, and the source's metadata records the OA recovery provenance.

---

## 15. Source vault and resumability

The source vault (`.ragent/research_vault/<run_tag>/`) persists raw captured source text and metadata in a SQLite index. Each vault entry records:

- Source URL, title, and fetch timestamp
- Search tool and engine that discovered the source
- Media type classifier (`page`, `pdf`, `youtube`, etc.)
- On-disk path to the raw content file
- Blake3 hash of the body text
- Full body text for full-text search

On a subsequent run with the same research name, the gatherer checks the vault before issuing new web searches. If the vault already contains enough sources to satisfy the requested tier's minimum, web search is skipped entirely. This avoids re-fetching already-captured sources and makes re-runs nearly instant.

**Vault skip thresholds**

| Tier | Minimum sources to skip web search |
|---|---|
| `light` | 3 |
| `full` | 8 |
| `dissertation` | 15 |

**Run manifest**

Each run also persists a **run manifest** (JSON) that tracks every pipeline step's lifecycle. On resume, the tier router reads the manifest and skips already-completed steps, restarting from the first pending or in-progress step. This means a interrupted `full`-tier run can be resumed without re-running earlier pipeline phases.

---

## 16. Templates

Place Markdown templates under `research/_templates/<name>.md`. They can use the placeholders `{{title}}`, `{{topic}}`, and `{{date}}`.

```
/research create quarterly "Q3 performance review" --template quarterly
```

The template becomes the skeleton and the standard sections are appended below it. Template placeholders are substituted:

- `{{title}}` — the research title (derived from the topic or seed source)
- `{{topic}}` — the research topic string
- `{{date}}` — the current date in ISO format

**Example template**

`research/_templates/quarterly.md`:

```
# {{title}}

**Period:** {{date}}

## Executive Overview

## Key Metrics

## Goals for Next Quarter
```

When used with `/research create quarterly "Q3 performance review" --template quarterly`, the template provides the skeleton and the standard research sections (findings, implications, references, etc.) are appended below the template content.

---

## 17. HTTP API

The research system is also accessible via HTTP endpoints. All endpoints are mounted under the auth-protected router.

### GET /research

List every research item (excludes archived by default).

**Response:**

```
{
    "items": [
        {
            "name": "rust-async",
            "title": "Rust async/await patterns",
            "status": "complete",
            "created_at": "2024-01-15T10:00:00Z",
            "modified_at": "2024-01-15T10:05:00Z",
            "sources": 12
        }
    ],
    "count": 1
}
```

### POST /research

Create and run a research session. Returns the research name, format, total source count, and all session events as a JSON array.

**Request body:**

```
{
    "name": "rust-async",
    "topic": "Rust async/await patterns",
    "title": "Optional custom title",
    "from_urls": ["https://example.com/article"],
    "from_file": "docs/design.md",
    "sources_dir": "docs/architecture",
    "template": "quarterly",
    "use_local": true,
    "use_specs": true,
    "use_low_relevance": false,
    "no_scholarly": false,
    "use_pdf": false,
    "fetch_concurrency": 10,
    "fetch_timeout_secs": 30,
    "local_concurrency": 8,
    "depth": "standard",
    "iterations": 3,
    "format": "report"
}
```

**Response (201 Created):**

```
{
    "name": "rust-async",
    "format": "report",
    "total_sources": 12,
    "events": [
        { "type": "phase", "phase": "setup" },
        { "type": "queries", "queries": ["..."] },
        { "type": "web", "url": "...", "title": "..." }
    ]
}
```

**Error codes:**

- `400 Bad Request` — invalid research name
- `409 Conflict` — research item already exists
- `500 Internal Server Error` — gathering or synthesis failure

#### GET /research/{name}

Show metadata for a single item, including related research items (top 5 by title similarity).

**Response:**

```
{
    "item": {
        "name": "rust-async",
        "title": "Rust async/await patterns",
        "status": "complete",
        "created_at": "2024-01-15T10:00:00Z",
        "modified_at": "2024-01-15T10:05:00Z",
        "sources": 12
    },
    "related": [
        {
            "name": "tokio-runtime",
            "title": "Tokio runtime internals",
            "snippet": "..."
        }
    ]
}
```

**Error codes:**

- `404 Not Found` — item not found (includes closest matches)
- `400 Bad Request` — invalid name

#### PUT /research/{name}

Replay the item's recorded invocation line: re-runs the full pipeline and overwrites `RESEARCH.md` and its associated files with freshly gathered results. This is the HTTP equivalent of `/research update`

<name> (and ragent research update <name>). The response uses the same envelope as POST /research.

```
curl -s -X PUT http://localhost:9100/research/rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

**Error codes:**

- 404 Not Found — item not found
- 409 Conflict — a run for this item is already in flight

#### DELETE /research/{name}

Delete a research item. Requires a confirmation token.

**Query parameter:** ?confirm=delete-{name}

The examples below assume a server started with ragent serve --addr 127.0.0.1:9100 (the default bind address is 127.0.0.1:3000).

```
curl -X DELETE http://localhost:9100/research/rust-async?confirm=delete-rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

**Error codes:**

- 412 Precondition Failed — missing or incorrect confirm token
- 404 Not Found — item not found

#### Example: full HTTP workflow

```
# Create a research item
curl -s -X POST http://localhost:9100/research \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"rust-async","topic":"Rust async/await patterns","use_local":true}' \
  | jq '.name, .total_sources'

# List all items
curl -s http://localhost:9100/research \
  -H "Authorization: Bearer $RAGENT_TOKEN" | jq '.items[] | .name'

# Show one item
curl -s http://localhost:9100/research/rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN" | jq '.item.title'

# Delete an item
curl -s -X DELETE "http://localhost:9100/research/rust-async?confirm=delete-rust-async" \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

---

## 18. CLI equivalents

The same commands work outside the TUI:

```
ragent research help
ragent research create rust-async "Rust async patterns" --tier full --use-local
ragent research create from-url --from-url https://example.com/article
ragent research create from-doc --from-file docs/design.md --use-local
ragent research create comp "vector db landscape" --mode competitive
ragent research open rust-async
ragent research list
ragent research list --all
ragent research search "vector database"
ragent research show rust-async
ragent research update rust-async
ragent research delete rust-async --yes
ragent research archive rust-async
ragent research continue rust-async "Also cover tokio vs async-std"
```

CLI `create` emits machine-readable JSON lines prefixed with `ragent-research:` so you can pipe them through `jq`:

```
ragent research create rust-async "Rust async patterns" 2>&1 | grep '^ragent-research:' | jq '.'
```

Each JSON line is a session event (phase, queries, web source captured, local source captured, iteration completed, critic result, etc.).

---

## 19. Configuration

Research-specific configuration lives under the `research` key in `ragent.json`:

```
{
    "research": {
        "open_access_recovery": true,
        "contact_email": "you@example.com",
        "oa_min_full_text_chars": 1000,
        "evaluate": {
            "enabled": true
        }
    }
}
```

| Field | Type | Default | Description |
|---|---|---|---|
| `open_access_recovery` | bool | `false` | Enable OA recovery via Unpaywall and Europe PMC |
| `contact_email` | string? | `null` | Email required by Unpaywall's ToS |
| `oa_min_full_text_chars` | usize | `1000` | Minimum body length that triggers OA recovery |

| Field | Type | Default | Description |
|---|---|---|---|
| `evaluate` | `ResearchEvaluateConfig` | `{"enabled": false}` | Self-evaluation scorecard settings (FR-015 of specs/opendeepresearch). When enabled, the pipeline appends a deterministic quality scorecard (quality, relevance, groundedness, completeness, structure) to the report. |

Web search engine keys are top-level in `ragent.json`:

```
{
    "langsearch_api_key": "...",
    "tavily_api_key": "...",
    "perplexity_api_key": "...",
    "exa_api_key": "...",
    "openalex_email": "you@example.com"
}
```

---

## 20. Tips for good results

- **Use specific topics.** Broad topics produce broad findings. Narrow questions yield more actionable reports. “Rust async cancellation semantics” is better than “Rust async”.
- **Pick the right tier.** `light` for quick lookups, `full` for decision documents, `dissertation` for long-form writing.
- **Pick the right depth.** `shallow` for fast orientation, `standard` for balanced research, `deep` for thorough investigation with more sources.
- **Enable `--use-local` and `--use-specs`** when the research should tie back to your own codebase or specifications.
- **Seed with `--from-url` or `--from-file`** when you want the report to start from a particular document.
- **Review the contradiction graph and source tensions** in `full` tier reports to spot conflicting evidence before acting on the findings.
- **Check `/websearch test`** if a run returns unexpectedly few web sources — a backend may be rate-limited or blocked.
- **Read the progress log** to see which sources were captured, which failed, and whether the synthesis used the LLM or the mechanical fallback.
- **Use `--no-papers`** for non-academic topics where scholarly results add noise.
- **Use `--use-pdf`** when PDFs are likely to contain the best evidence (white papers, technical reports, academic papers).
- **Use `--use-low-relevance`** when researching niche topics where even low-relevance sources may contain useful information.
- **Use `--web-time`** to raise (or lower) the web phase deadline — by default the web phase is capped at 60 seconds, after which everything gathered so far is ingested and the run continues to analysis/synthesis. Set `--web-time 0` to remove the cap on slow networks. Once the deadline elapses,

no new search or fetch is started; the only overshoot is the completion of fetches already in flight, each capped by `--fetch-timeout-secs` (default 30 s), so the phase always returns.

- **Re-run with the same name** to leverage the source vault — if enough sources are already cached, web search is skipped and the run is nearly instant.
- **Use templates** for recurring report formats (quarterly reviews, security assessments, etc.) to ensure consistent structure.
- **Use `--format comparison-table`** when evaluating competing options — the comparison table layout makes trade-offs visible at a glance.
- **Use `--format imrad`** for scientific or technical reports that need the Introduction/Methods/Results/Discussion structure.

---

## 21. Troubleshooting

| Symptom | Likely cause | What to do |
|---|---|---|
| `0 sources` or very few | Search backends blocked / rate-limited | Run `/websearch test`; wait or add an API-backed engine |
| `LLM synthesis failed - using mechanical fallback` | No API key, model down, or output malformed | Check provider setup; the report still contains deterministic summaries |
| `CITATION_VERIFICATION_FAILED` | A citation marker `[#N]` points to a missing source | Run again with `--tier full`; the gate blocks shipment |
| `web phase deadline reached` | The 60-second web phase deadline passed (slow network or many candidate pages) | The run continues with the sources gathered so far; raise `--web-time N` or lower `--fetch-concurrently` for fuller coverage. The deadline stops new searches and fetches from being started, so the phase cannot run unbounded — at most the fetches already in flight finish (each capped by `--fetch-timeout-secs`) |
| `local phase timed out` | Large project with many files | Raise `--local-phase-timeout-secs` or lower `--local-concurrently` |
| Name rejected | Invalid research name | Use 3–64 lowercase ASCII letters/digits/hyphens starting with a letter |
| `research/<name>` already exists | Duplicate create | Use `/research open <name>` or pick a new name |
| Very slow run | Deep depth + full tier + many sources | Use `--depth standard` or `--tier light`; reduce `--fetch-concurrently` |

| Symptom | Likely cause | What to do |
|---|---|---|
| No scholarly sources | `--no-papers` is set or OpenAlex is down | Remove `--no-papers`; check `/websearch test` for OpenAlex status |
| OA recovery not working | Not configured or `contact_email` missing | Set `research.open_access_recovery: true` and `contact_email` in `ragent.json` |
| PDFs skipped | `--use-pdf` not set | Add `--use-pdf` to allow PDF web sources |
| Sources not reused on re-run | Vault threshold not met | The vault needs 3/8/15 sources (light/full/dissertation) before skipping web search |

---

## 22. End-to-end examples

**Example 1: Technology evaluation**

Research and compare three vector databases for a RAG system:

```
/research create vector-db-eval "Comparison of pgvector, Qdrant, and Weaviate for RAG application
```

This produces a comparison-table report with each database evaluated across multiple dimensions (performance, scalability, ease of use, ecosystem), gathered from web sources and cross-referenced with any local project files.

**Example 2: Academic paper review**

Research a specific paper using the IMRaD format:

```
/research create attention-paper "Attention Is All You Need - Transformer architecture analysis"
```

This seeds the research from the arxiv page, derives the topic from the abstract, gathers related sources, and produces an IMRaD-structured report.

**Example 3: Project architecture audit**

Audit your own codebase architecture with web context:

```
/research create arch-audit "Analysis of current project architecture patterns and improvement op
```

This scans project files under `src/`, cross-references existing specs, and supplements with web research on architecture best practices.

**Example 4: Quick competitive intelligence**

Fast check on a competitor's latest release:

```
/research create competitor-check "Latest features in Kubernetes 1.30" --tier light --depth shall
```

This produces a quick summary without the full adversarial pipeline.

**Example 5: Long-form survey**

Comprehensive survey for a thesis or white paper:

```
/research create multi-agent-survey "Survey of multi-agent orchestration frameworks and patterns"
```

This partitions the topic into chapters, runs the full pipeline per chapter, and produces a long-form deliverable.

**Example 6: Seeded from a local design document**

Research starting from a local design doc, augmented with web sources:

```
/research create design-review --from-file docs/architecture/design.md --use-local --use-specs --
```

The design document is extracted as the primary source, the topic is derived from its content, and web search supplements with external context.

---

*End of manual.*